Skills Battle 2026

Application Development

Version 1.1

Sudoku

1. Project Introduction

In addition to the classic Sudoku, there are various tricky variants. One variant is to fill in the squares based on a set of sums over several squares. The same Sudoku rules apply to the solution to be found.

The objective is to fill the grid with numbers from 1 to 9 in a way that the following conditions are met:

- Each row, column, and nonet contains each number exactly once.
- The sum of all numbers in a cage must match the small number printed in its corner.
- No number appears more than once in a cage.
- The solution must be unique.

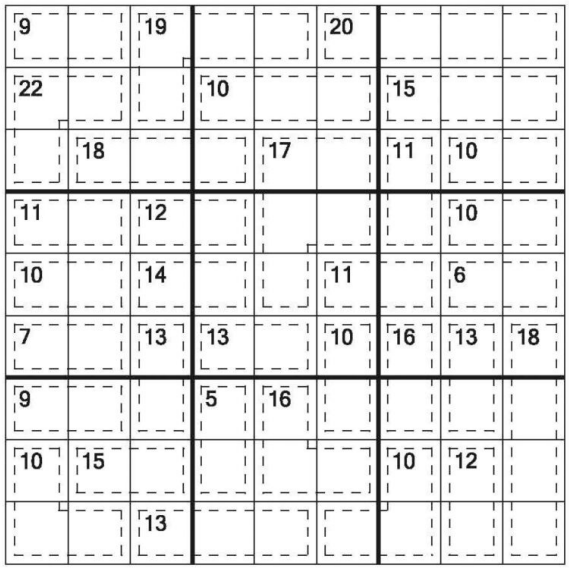

Figure 1: Example

Your task is to plan, implement, and test an application based on a set of given use cases. To test your creativity and planning, you will only receive the use case diagram. Designing and implementing the user interface and user experience are part of your task.

1.1 Design

You are free to design the application. Your task is to create mockups. The mockups are part of your documentation and have to be handed in. Add an appropriate navigation to your interface. This can be a menu or a set of buttons. Add controls for all functions you have to implement.

1.2 Examples

You get 2 examples to test your application. If needed, you can generate more examples online.

1.3 Database

Please create a database named “sudoku” on MySQL or MS-SQL server running on your machine. You are free to build the database schema.

Create for each table a SQL statement and save them in a script called sudoku.sql. Don’t forget to create the primary and foreign key constraints.

1.4 Deliverables

All deliverables must be submitted in a zip file named "AppDev_Name_FirstName.zip". The deliverables are the documentation including the test protocol, executable files, the source code and the database script.

Only the contents of the zip file will be considered for the evaluation.

Note that the submission of the planned test cases must take place by 12 o'clock.

Upload folder (OneDrive)

2. Features

2.1 Use Cases

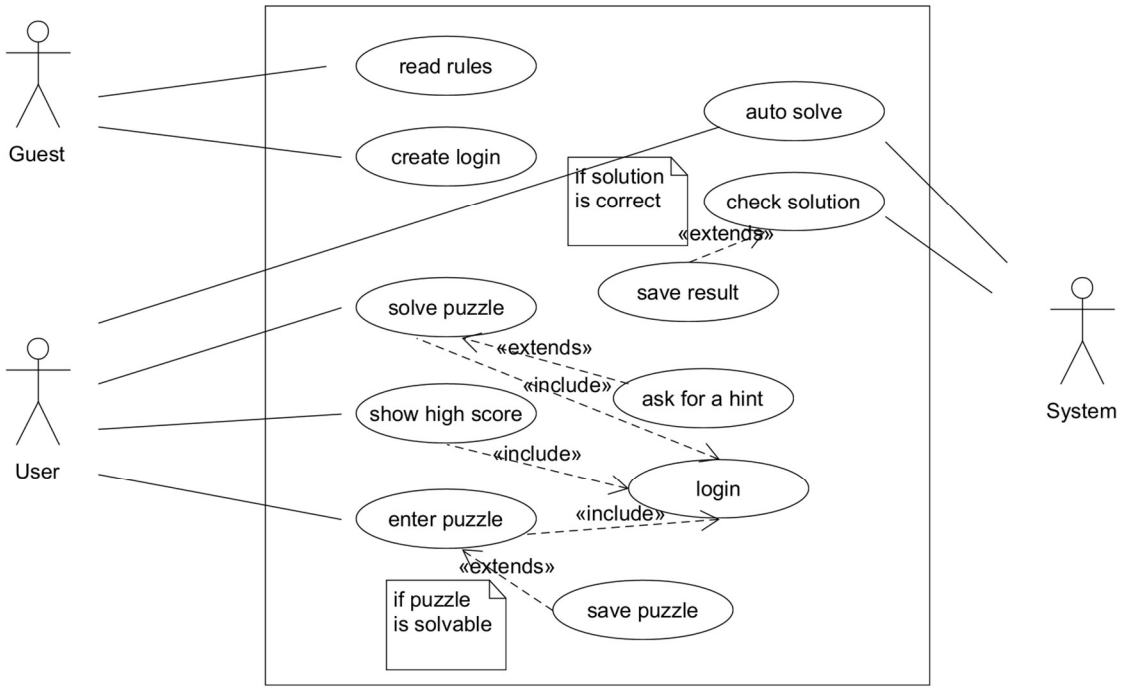

Figure 2: Use case diagram

Use case 1: read rules

On the start page, an example as well as the rules of the game must be displayed.

Use case 2: create user

To use the application, a login is required. Define the necessary fields in the mockups and in the database.

Use case 3: login

The user logs in.

Use case 4: enter puzzle

The user can manually enter a puzzle, which can be solved by other users. The creator should be able to specify the difficulty level from 1-3. The puzzle should be stored in the database. No solution is recorded. Solutions must be calculated with an algorithm.

Use case 5: save new puzzle

The puzzle can only be saved if it is solvable. Check with an algorithm if the puzzle can be solved before saving.

Use case 6: solve puzzle

Every user can solve stored puzzles.

Use case 7: ask for a hint

If the user is stuck, they can ask for a hint. This should be considered for the high score at the end. Develop a suggestion for the hint, document it, and implement your solution.

Use case 8: show high score

Implement rules for a high score based on the time needed and the number of hints used.

Use case 9: check solution

If all fields are filled, check the solution.

Use case 10: save result

Save the time required and the number of hints used in the database.

Use case 11: auto solve

Implement the possibility to automatically solve the Sudoku. You will need this function for the hints and for checking a new puzzle.

2.3 Additional use cases

Add additional use cases to the use case diagram. Think of 3 additional use cases that you can implement. Choose them based on the existing entries. Write done your decision in the documentation.

2.3 Validation

Validation is an essential part of every application. Define based on the requirements validation rules for all user inputs and user interactions.

The following validation to check if the puzzle is correct is given:

Calculate the sum of all numbers in the completed Sudoku. The value can be determined unambiguously. Note your calculation in the documentation. Use the value for a "simple" validation before checking the solution with an algorithm and saving the captured puzzle.

2.4 Mockups

Plan your user interface. Create mockups in your tool of choice. Add the design to your documentation.

2.5 Implementation

Implement your application based on the use cases, your screen design, and the validation rules.

2.6 Documentation

Create a documentation that includes the following sections:

- Mockup
- Database diagram
- Class diagram
- Additionally chosen use cases
- Validation rules
- Test protocol

You can create the documentation as a Word file, Readme file, etc. For the delivery, create a PDF document.

3. Testing

3.1 Additional use cases

Plan your test cases before you start the implementation. For each use case, create at least one positive, one negative, and, where possible, one test case for boundary conditions. If possible, choose test cases that you can implement as a unit test. You can save the test cases in a Word document, Excel, or similar.

Note that the submission of the planned test cases must take place by 12 o'clock.

3.2 Additional use cases

Running the test cases should be implemented with a test framework and is part of the submission (e.g. JUnit in Java).

UI tests can be run without a test framework.

4. Delivery

Please hand in your deliverables (see chapter 1.4).